Entraîner un réseau de neurones artificiels (partie 1)

CSI 4506 - automne 2026

Marcel Turcotte
Version : 27 oct. 2026 10h55

Préambule

Message du jour

https://youtu.be/FBfNXq_R3oo

Benaich, N. et Air Street Capital (2026). *State of AI Report 2026*. Paru le 8 octobre 2026. Regarder la vidéo d'accompagnement.

Le *State of AI Report 2026*, de Nathan Benaich et Air Street Capital, fait le point sur les avancées de l'IA en recherche, dans l'industrie, en matière de politiques publiques et de sécurité. La vidéo d'accompagnement donne un aperçu du rapport.

Pendant que nous étudions l'entraînement des réseaux de neurones, ce rapport relie les méthodes sous-jacentes au domaine dans son ensemble : les progrès des capacités des modèles, l'infrastructure informatique qui les soutient et leurs applications. Le rapport et la vidéo sont des ressources complémentaires pour explorer ce contexte plus large.

Objectifs d'apprentissage

- **Suivre** les calculs d'un réseau à propagation avant dans son implémentation NumPy.
- **Distinguer** les probabilités prédites, les étiquettes de classe et la perte d'entraînement.
- **Expliquer** comment un algorithme naïf essaie des modifications de paramètres et applique des mises à jour simultanées.
- **Identifier** les limites de calcul qui motivent l'introduction de la rétropropagation.

MLP naïf

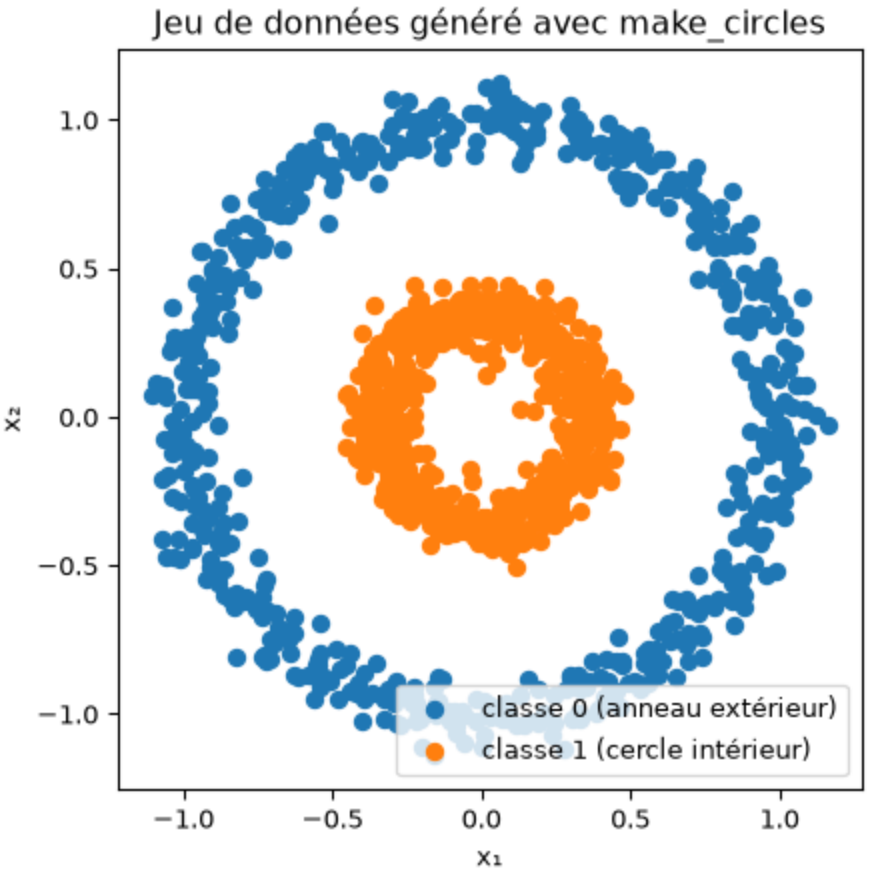
Données

In [2]:
```python
# Générer et tracer le jeu de données circulaire
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Générer des données synthétiques
X, y = make_circles(n_samples=1200, factor=0.35, noise=0.06, random_state=42

# Séparer les coordonnées pour le tracé
x1, x2 = X[:, 0], X[:, 1]

# Tracer les deux classes
plt.figure(figsize=(5, 5))
plt.scatter(x1[y==0], x2[y==0], color="C0", label="classe 0 (anneau extérieu
plt.scatter(x1[y==1], x2[y==1], color="C1", label="classe 1 (cercle intérieu
plt.xlabel("x₁")
plt.ylabel("x₂")
plt.title("Jeu de données généré avec make_circles")
plt.axis("equal") # conserver les proportions des cercles
plt.legend()
plt.show()
```



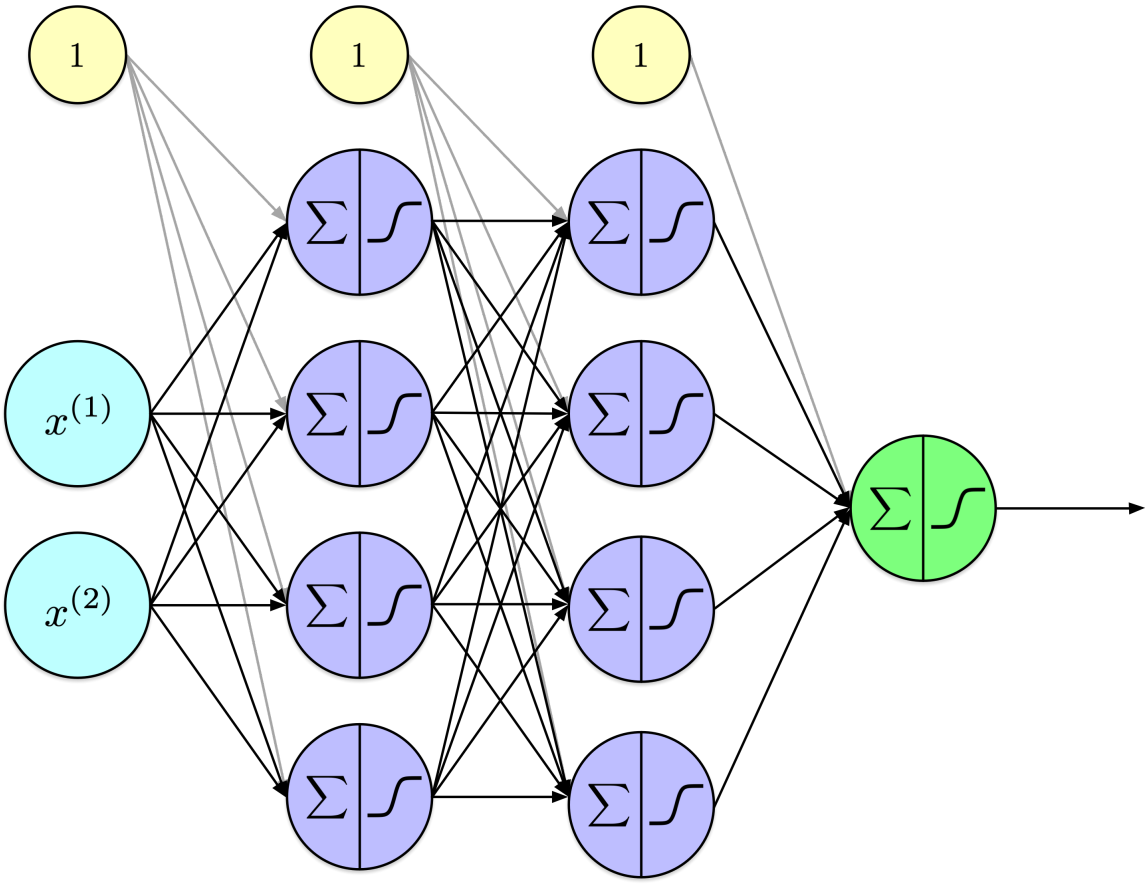
La présentation 11 a établi comment un réseau dont les paramètres sont fixés calcule des prédictions. Il s'agit maintenant de choisir ces paramètres à partir des données. Une

implémentation NumPy complète rend le calcul et l'entraînement accessibles par la programmation, avant l'introduction de la rétropropagation.

Le jeu de données comprend un anneau extérieur (classe 0) et un cercle intérieur (classe 1). Ces classes ne sont pas linéairement séparables dans l'espace d'attributs d'origine (x_1, x_2).

L'algorithme d'entraînement essaie des modifications de paramètres sans calculer de dérivées. Il expose la boucle d'entraînement et le coût des évaluations répétées du réseau. La rétropropagation fournira une méthode efficace de calcul des gradients dans la prochaine présentation.

Architecture



Nous implémentons le réseau de neurones ci-dessus : une couche d'entrée de 2 unités, deux couches cachées de 4 unités chacune et une couche de sortie d'une unité. Nous notons cette architecture [2, 4, 4, 1].

Notre réseau, un perceptron multicouche (*multilayer perceptron*, MLP), possède deux nœuds d'entrée correspondant aux deux attributs du jeu de données. Les expériences avec TensorFlow Playground et Keras, décrites dans Séparabilité de données circulaires, ont montré qu'une configuration de deux couches cachées de quatre neurones chacune permet de classer les exemples.

Le réseau possède **trois couches paramétrées** : deux couches cachées de quatre unités chacune et une couche de sortie d'une seule unité. Les deux nœuds d'entrée transmettent les attributs au réseau et ne sont pas comptés comme des couches paramétrées.

Dans cette implémentation, chaque unité de calcul utilise la fonction d'activation sigmoïde. La sortie finale estime la probabilité de la classe 1. Utiliser la même activation partout garde le code de propagation avant concis; les couches cachées ne sont pas obligées d'utiliser la sigmoïde en général.

Dans le schéma, les arêtes noires entre les unités représentent les poids du modèle. Trois nœuds produisant la constante 1 servent uniquement à représenter les biais; ils ne correspondent pas à des nœuds explicitement implémentés dans le modèle. Les arêtes grises qui les relient aux autres unités représentent les termes de biais.

Fonctions utilitaires

In [3]:
```python
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def bce_loss(y_true, y_prob, eps=1e-9):

    """Perte d'entropie croisée binaire (moyenne sur les données)."""

    y_prob = np.clip(y_prob, eps, 1 - eps)

    return -np.mean(y_true * np.log(y_prob) + (1 - y_true) * np.log(1 - y_pr
```

Nous définissons deux fonctions utilitaires, `sigmoid` et `bce_loss`. La perte d'entropie croisée binaire (*binary cross-entropy*, BCE) a la même définition que dans notre modèle de régression logistique. Vous souvenez-vous d'un autre nom donné à cette perte?

Dans `bce_loss`, les probabilités sont bornées à l'intervalle $[\epsilon, 1-\epsilon]$. Ce bornage (*clipping*) évite les expressions logarithmiques $\log(0)$ et $\log(1-1) = \log(0)$, et prévient ainsi les valeurs indéfinies ou infinies pendant le calcul.

Les deux fonctions acceptent des tableaux NumPy (`ndarray`). En particulier, `bce_loss` calcule la perte sur l'ensemble du jeu de données sans boucle explicite sur les exemples.

NaiveMLP

L'implémentation complète est présentée ci-dessous; les diapositives suivantes en examinent les différentes parties.

In [4]:

```python
class NaiveMLP:

    """
    Un MLP minimal pour la classification binaire, entraîné par un algorithm
    naïf sans dérivées qui applique les mises à jour simultanément.

    Cette méthode volontairement simple illustre le processus d'entraînement
    et son coût de calcul avant l'introduction de la rétropropagation.
    """

    def __init__(self, layer_sizes, step=0.1, seed=None):

        self.sizes = list(layer_sizes)
        self.step = float(step)
        rng = np.random.default_rng(seed)

        # Initialiser les poids et les biais

        self.W = [rng.standard_normal(size=(in_d, out_d)) * 0.5
                  for in_d, out_d in zip(layer_sizes[:-1], layer_sizes[1:])]

        self.b = [np.zeros(out_d) for out_d in layer_sizes[1:]]

    def forward(self, X):

        """
        Propagation avant simple : calculer uniquement les activations de sc

        X : dimensions (N, input_dim)

        Retour : probabilités de sortie, dimensions (N,)
        """

        a = X
        for W, b in zip(self.W, self.b):
            a = sigmoid(a @ W + b)

        return a.ravel()

    def predict(self, X, threshold=0.5):

        return (self.forward(X) >= threshold).astype(int)

    def loss(self, X, y):

        return bce_loss(y, self.forward(X))

    def _all_param_tags(self):

        """
        Produit des repères pour chaque paramètre scalaire :
        ('W', layer_idx, i, j) ou ('b', layer_idx, j)
        """

        for l, W in enumerate(self.W):
```

```
            for i in range(W.shape[0]):
                for j in range(W.shape[1]):
                    yield ('W', l, i, j)
            for j in range(self.b[l].shape[0]):
                yield ('b', l, j)

    def _get_param(self, tag):
        kind = tag[0]
        if kind == 'W':
            _, l, i, j = tag
            return self.W[l][i, j]
        else:
            _, l, j = tag
            return self.b[l][j]

    def _set_param(self, tag, val):
        kind = tag[0]
        if kind == 'W':
            _, l, i, j = tag
            self.W[l][i, j] = val
        else:
            _, l, j = tag
            self.b[l][j] = val

    def train(self, X, y, epochs=10, verbose=True):

        """
        Mise à jour simultanée :
        - Pour chaque paramètre scalaire θ, essayer θ + δ avec δ dans {-step
          retenir le δ donnant la perte minimale.
        - Recueillir tous les δ retenus, puis appliquer toutes les mises à j
        """

        for ep in range(1, epochs + 1):

            base_loss = self.loss(X, y)
            updates = {}

            # Essayer des modifications de tous les paramètres
            for tag in self._all_param_tags():

                theta = self._get_param(tag)
                best_delta = 0.0
                best_loss = base_loss

                for delta in (-self.step, 0.0, +self.step):
                    self._set_param(tag, theta + delta)
                    trial_loss = self.loss(X, y)
                    if trial_loss < best_loss:
                        best_loss = trial_loss
                        best_delta = delta

                # rétablir la valeur d'origine
                self._set_param(tag, theta)
                updates[tag] = best_delta
```

```
            # Appliquer toutes les modifications ensemble
            for tag, d in updates.items():
                if d != 0.0:
                    self._set_param(tag, self._get_param(tag) + d)

            new_loss = self.loss(X, y)

            if verbose:
                print(f"Époque {ep:3d} : perte {base_loss:.5f} → {new_loss:.
```

La classe complète est incluse ici pour que le cahier Jupyter puisse être exécuté de manière autonome.

Définition de la classe

```
class NaiveMLP:

    """
    Un MLP minimal pour la classification binaire, entraîné par un
algorithme
    naïf sans dérivées qui applique les mises à jour simultanément.

    Cette méthode volontairement simple illustre le processus
d'entraînement
    et son coût de calcul avant l'introduction de la
rétropropagation.
    """
```

Constructeur

```
    def __init__(self, layer_sizes, step=0.1, seed=None):

        self.sizes = list(layer_sizes)
        self.step = float(step)
        rng = np.random.default_rng(seed)

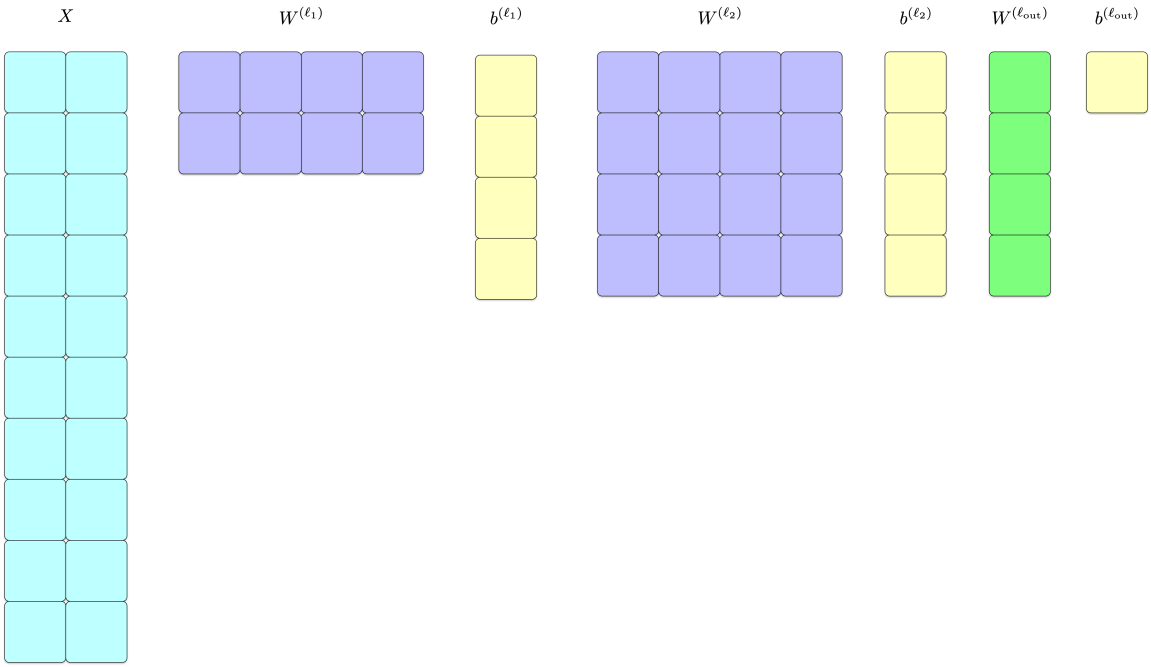
        # Initialiser les poids et les biais

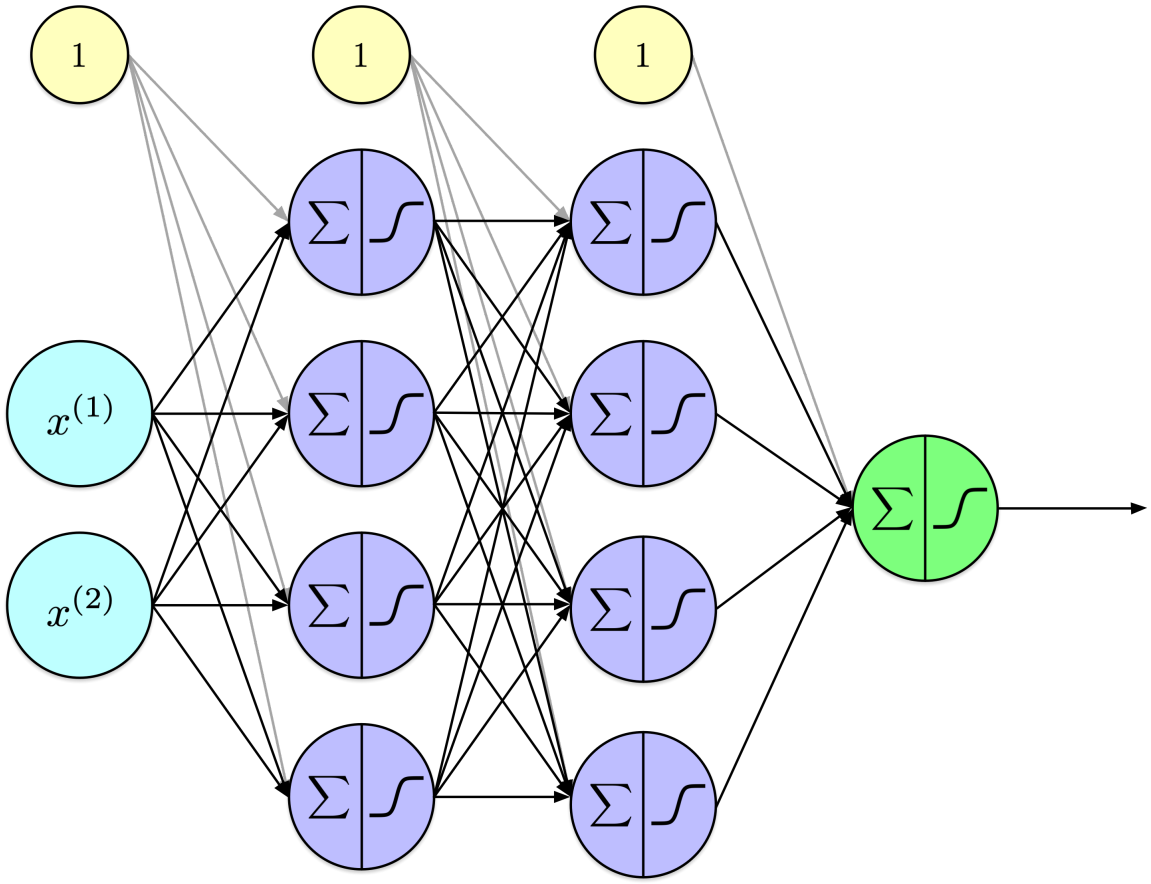
        self.W = [rng.standard_normal(size=(in_d, out_d)) * 0.5
                  for in_d, out_d in zip(layer_sizes[:-1],
layer_sizes[1:])]

        self.b = [np.zeros(out_d) for out_d in layer_sizes[1:]]
```

Conserver le nombre de couches, le nombre d'unités par couche et la taille du pas de l'algorithme d'apprentissage. Initialiser les poids (W) et les biais (b).

Paramètres





Dans notre exemple fil conducteur, le modèle comporte trois couches : $\ell^{(1)}$, $\ell^{(2)}$ et $\ell^{(\text{out})}$. Chaque couche ℓ est paramétrée par une matrice de poids $W^{(\ell)}$ et un vecteur de biais $b^{(\ell)}$. Le jeu de données X contient 10 exemples, chacun comportant deux attributs.

Assurez-vous de comprendre comment les dimensions d'entrée et de sortie de chaque couche déterminent les dimensions de sa matrice de poids et de son vecteur de biais.

Détour Python : tableaux de poids

In [5]:
```
seed = 0

rng = np.random.default_rng(seed)

layer_sizes = [2, 4, 4, 1]

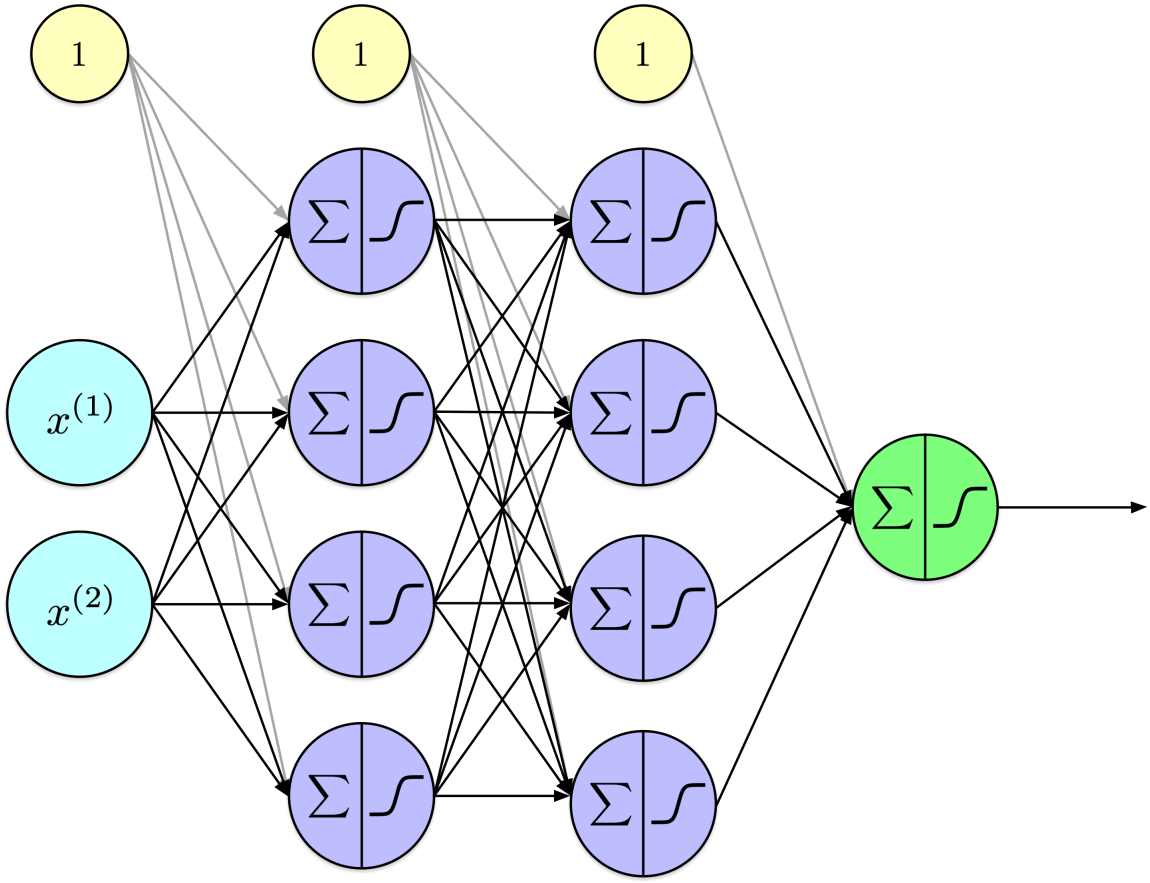
[(in_d, out_d) for in_d, out_d in zip(layer_sizes[:-1], layer_sizes[1:])]
```

```
[(2, 4), (4, 4), (4, 1)]
```

...

In [6]:
```
[rng.standard_normal(size=(in_d, out_d)) * 0.5 for in_d, out_d in zip(layer_
```

```
[array([[ 0.06286511, -0.06605243,  0.32021133,  0.05245006],
        [-0.26783469,  0.18079753,  0.65200002,  0.47354048]]),
 array([[-0.35186762, -0.63271074, -0.31163723,  0.02066299],
        [-1.16251539, -0.10939583, -0.62295547, -0.36613368],
        [-0.27212949, -0.15815008,  0.20581527,  0.52125668],
        [-0.06426733,  0.68323174, -0.33259734,  0.17575504]]),
 array([[ 0.45173509],
        [ 0.04700615],
        [-0.37174962],
        [-0.46086269]])]
```



Combien de poids ce réseau possède-t-il?

Détour Python : tableaux de biais

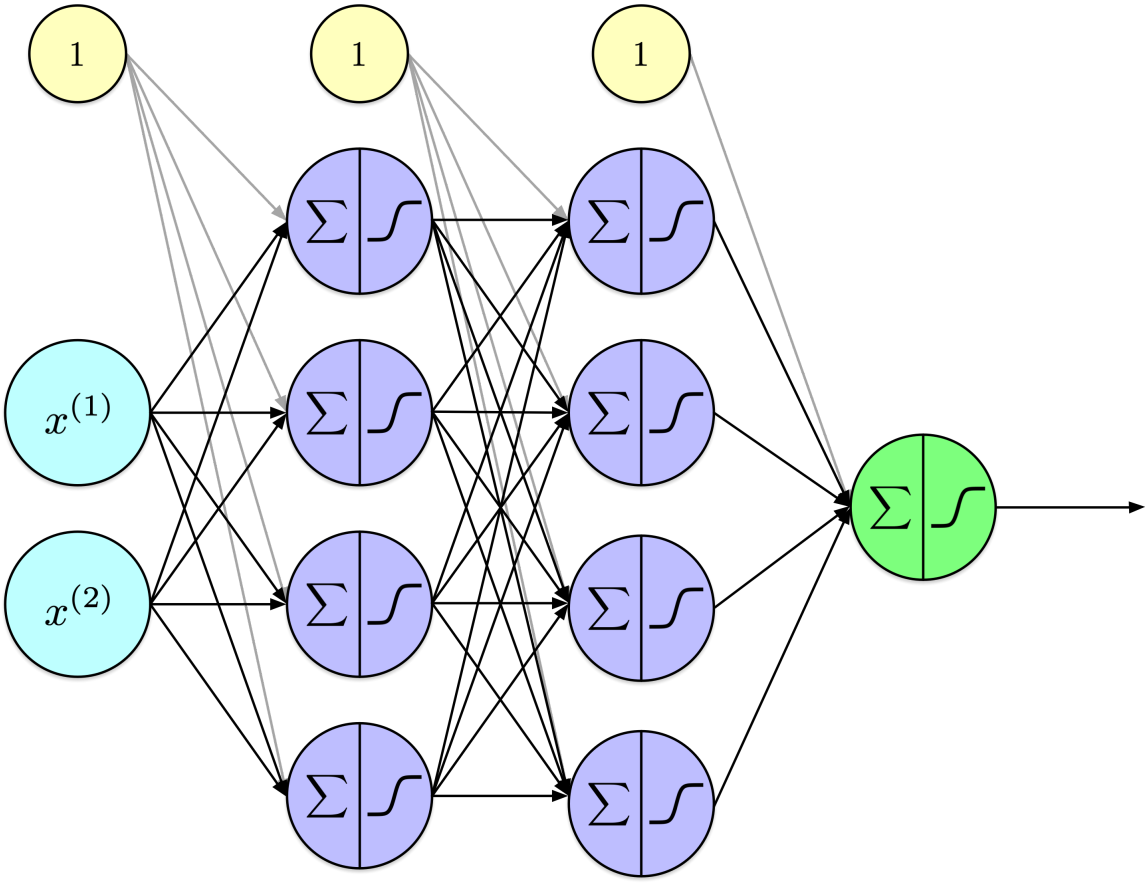
In [7]:
```
[out_d for out_d in layer_sizes[1:]]
```

```
[4, 4, 1]
```

...

In [8]:
```
[np.zeros(out_d) for out_d in layer_sizes[1:]]
```

```
[array([0., 0., 0., 0.]), array([0., 0., 0., 0.]), array([0.])]
```



Combien de termes de biais ce réseau possède-t-il?

Le réseau possède 37 paramètres au total : 28 poids et 9 termes de biais.

Propagation avant

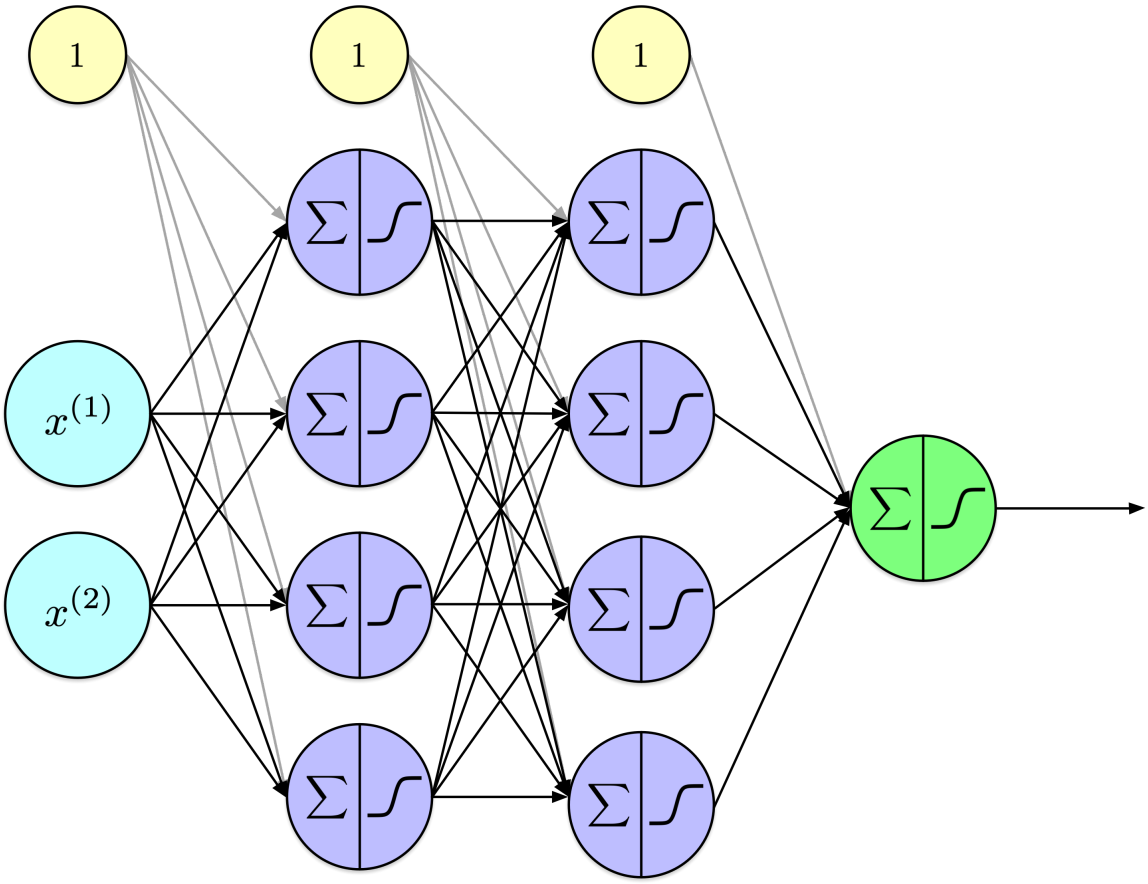
Retour au réseau : calculer les sorties de chaque couche à partir de la couche précédente.

```
    def forward(self, X):

        """
        Propagation avant simple : calculer les activations de
sortie.
        X : dimensions (N, input_dim)
        Retour : probabilités de sortie, dimensions (N,)
        """

        a = X
        for W, b in zip(self.W, self.b):
            a = sigmoid(a @ W + b)

        return a.ravel()
```



Calculer les activations de sortie, c'est-à-dire la probabilité que chaque exemple du jeu de données X appartienne à la classe 1.

La méthode procède séquentiellement, couche par couche. Notre exemple comporte trois couches de calcul distinctes.

Que sont `W` et `b` ?

`W` et `b` sont des listes de trois éléments chacune. La liste `W` contient des matrices de poids de dimensions 2×4, 4×4 et 4×1, tandis que `b` contient des tableaux de biais de tailles 4, 4 et 1, respectivement.

À quoi sert `zip(self.W, self.b)` ?

Cette fonction associe chaque matrice de poids à son tableau de biais, produisant une paire pour chacune des trois couches paramétrées.

Que représente `X` ?

L'argument `X` contient N exemples en lignes et deux attributs en colonnes; ses dimensions sont donc `(N, 2)`. Dans la partition d'entraînement des cercles, $N = 900$. Chaque itération de la boucle calcule les sorties de la couche courante pour tous les exemples à la fois.

Le code utilise `a` pour les activations : ce sont les sorties des unités notées o dans la présentation 11. L'expression `a @ W + b` calcule les préactivations, puis `sigmoid(...)` produit les activations suivantes. Enfin, `ravel()` transforme l'unique colonne de sortie en un tableau unidimensionnel de probabilités. Cette classe suppose une seule unité de sortie.

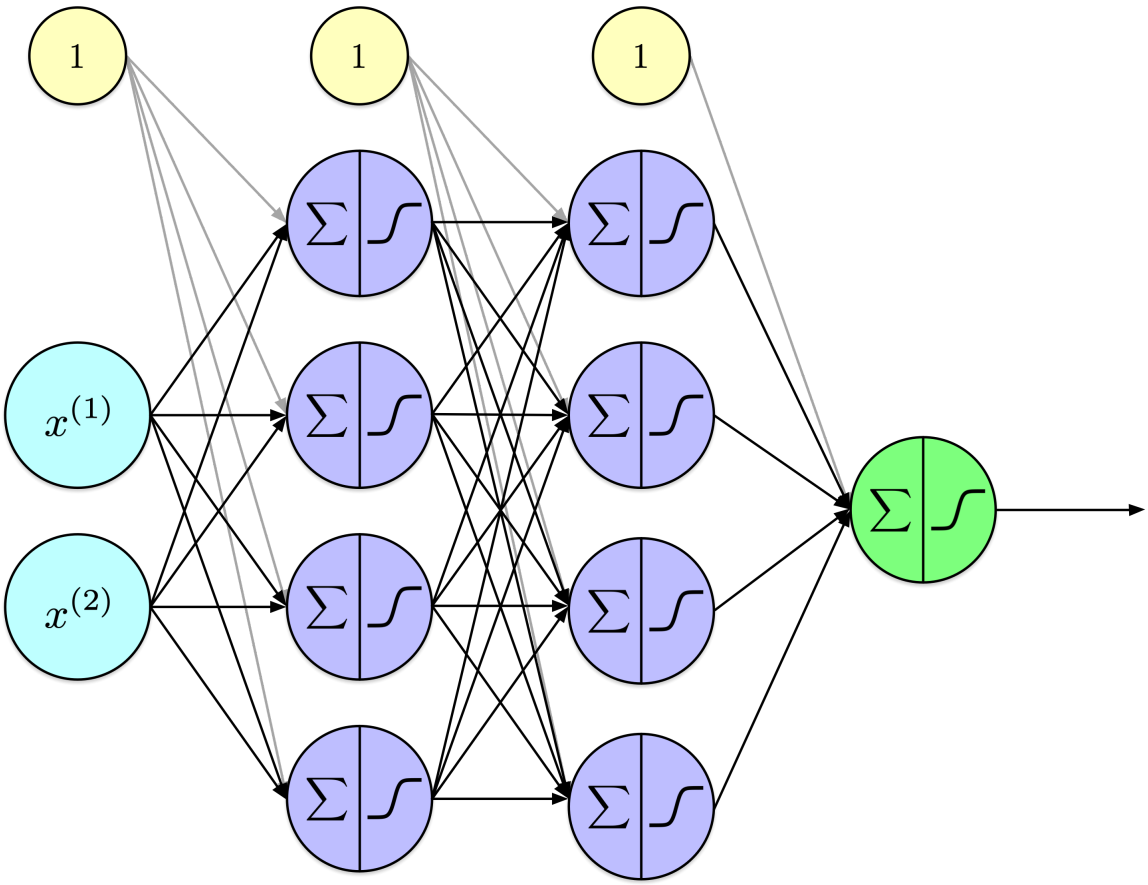
Suivre les dimensions des tableaux

Pour `layer_sizes = [2, 4, 4, 1]`, chaque ligne reste un exemple :

Couche	`a` en entrée	`W`	`b`	`a` en sortie
Cachée 1	`(N, 2)`	`(2, 4)`	`(4,)`	`(N, 4)`
Cachée 2	`(N, 4)`	`(4, 4)`	`(4,)`	`(N, 4)`
Sortie	`(N, 4)`	`(4, 1)`	`(1,)`	`(N, 1)`

```
a = sigmoid(a @ W + b)
```

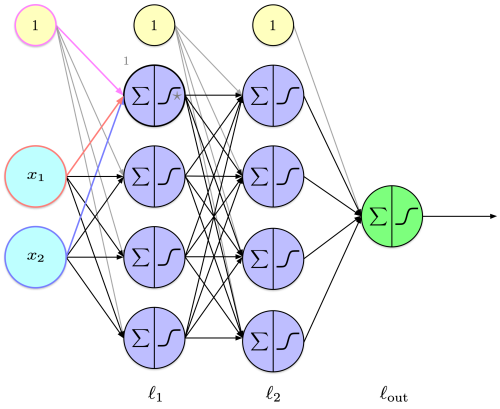
- `@` combine chaque exemple avec les poids de la couche.
- La **diffusion** (*broadcasting*) ajoute le même vecteur de biais à chaque ligne.
- `ravel()` change les dimensions finales de `(N, 1)` à `(N,)`.

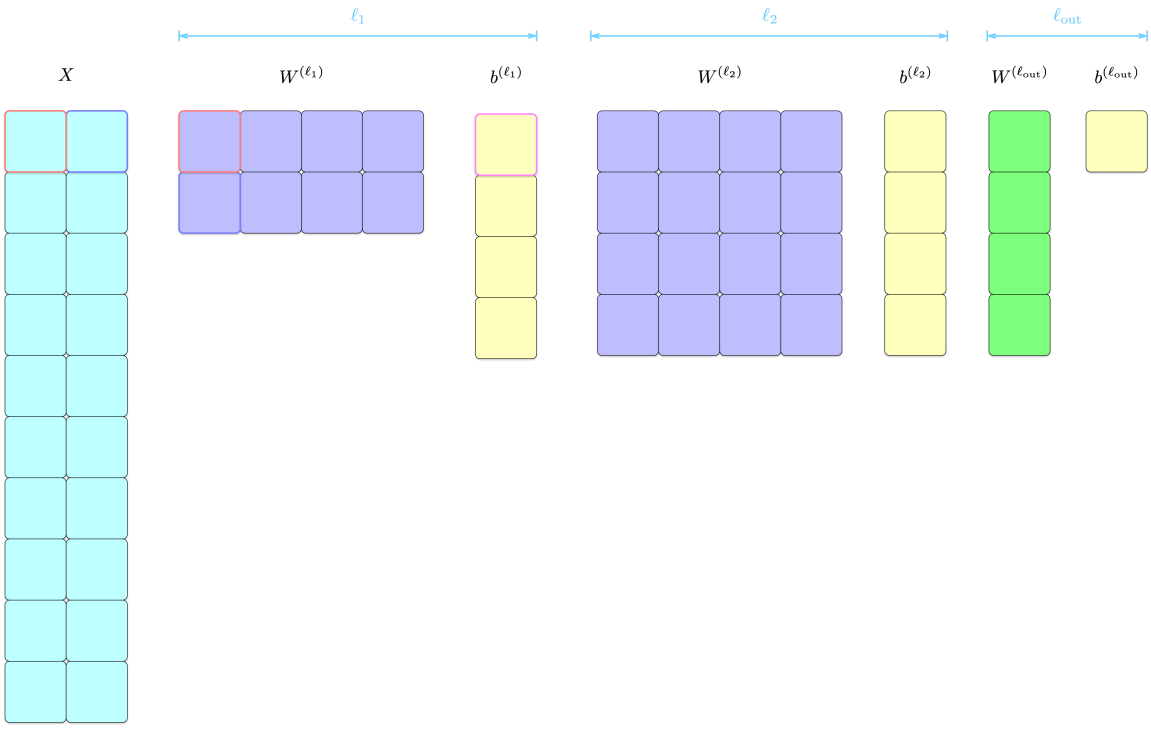


La multiplication matricielle exige que le nombre de colonnes des activations entrantes corresponde au nombre de lignes de la matrice de poids. Le nombre de colonnes du résultat est le nombre d'unités de la couche courante. L'application de la sigmoïde élément par élément conserve ces dimensions.

La diffusion ne crée pas de biais entraînable distinct pour chaque exemple. Tous les exemples utilisent les mêmes quatre biais dans une couche cachée de quatre unités, ou le même biais unique dans la couche de sortie. Le réseau possède donc 28 poids et 9 biais, soit 37 paramètres entraînables.

Suivre les dimensions (suite)



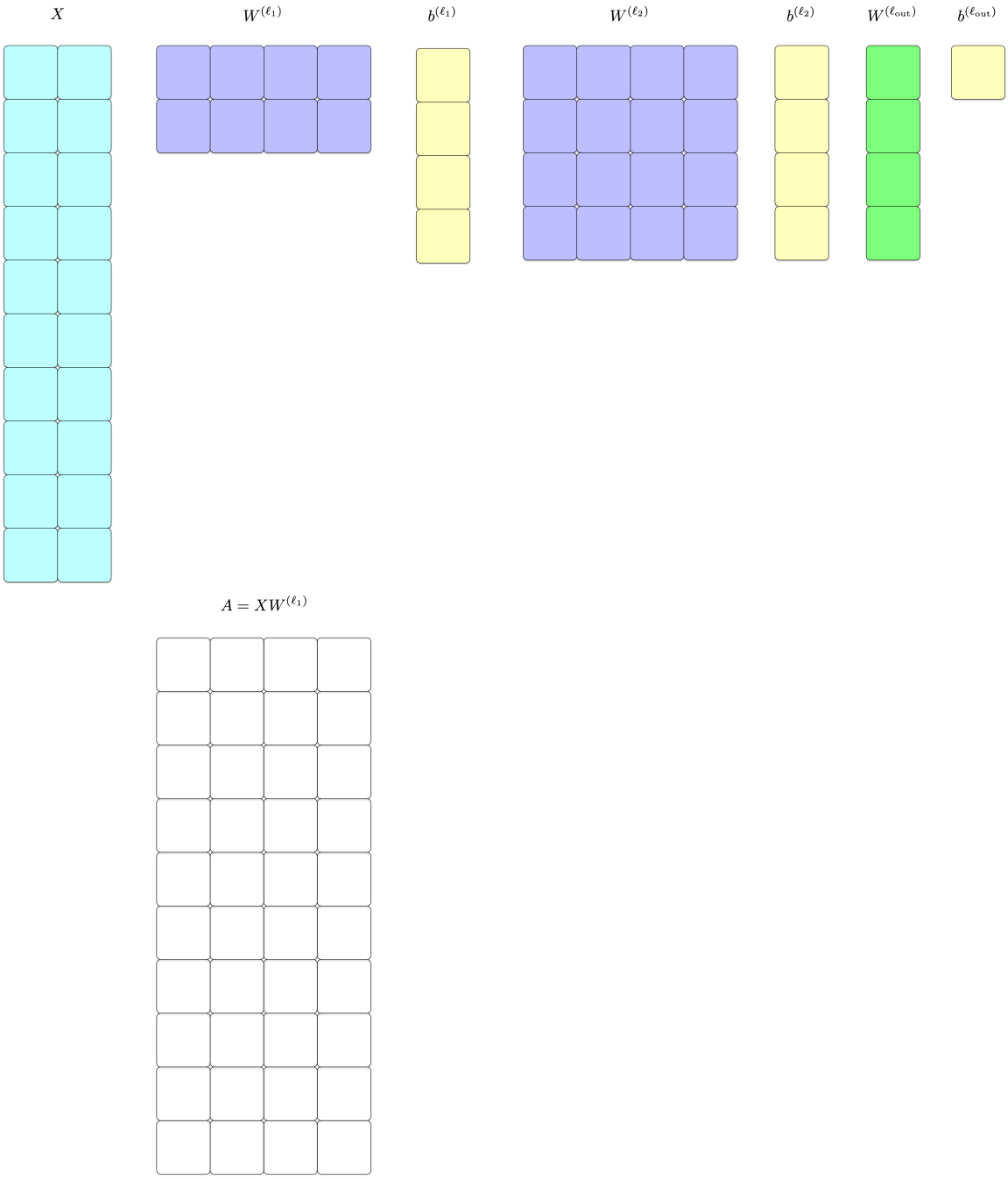


La figure représente le modèle, y compris le vecteur d'entrée X, les matrices de poids et les vecteurs de biais. Pour le neurone 1 de la couche ℓ_1, la valeur de préactivation est calculée comme suit :

$$z_1^{(\ell_1)} = x_1 W_{1,1}^{(\ell_1)} + x_2 W_{1,2}^{(\ell_1)} + b_1^{(\ell_1)}.$$

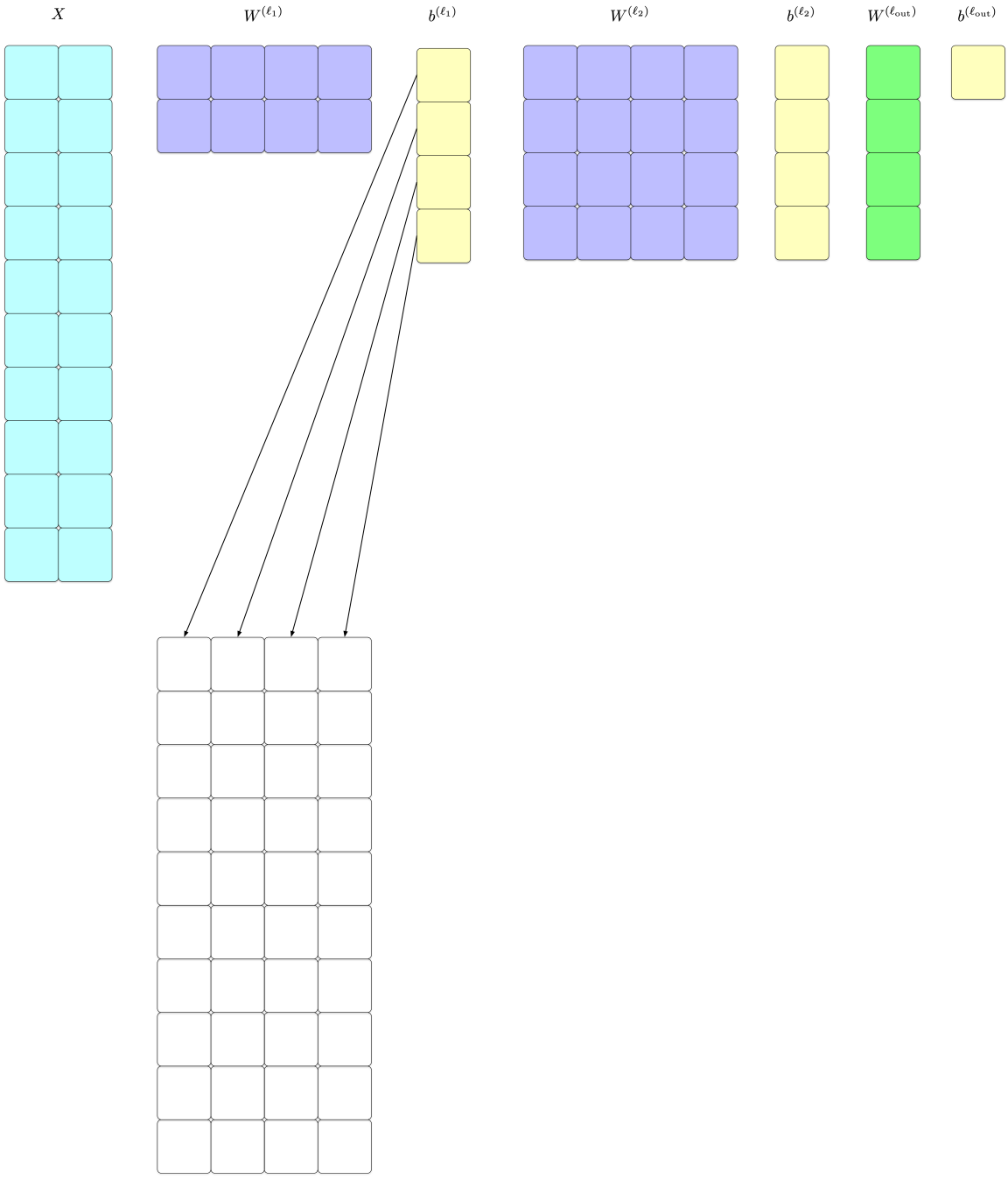
L'indexation commence à 1 dans l'ensemble du document pour faciliter la lecture.

Suivre les dimensions (suite)



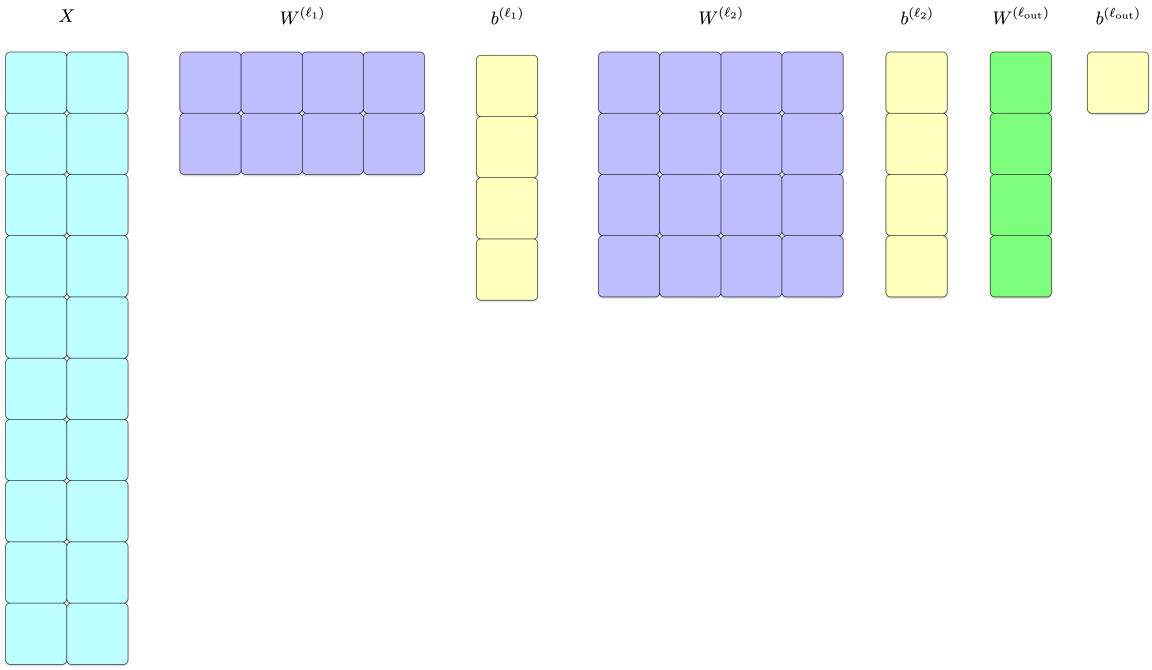
La figure illustre les dimensions de la matrice obtenue par le calcul intermédiaire $A = XW^{(\ell_1)}$.

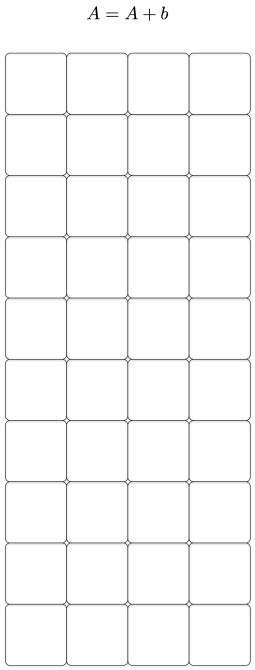
Suivre les dimensions (suite)



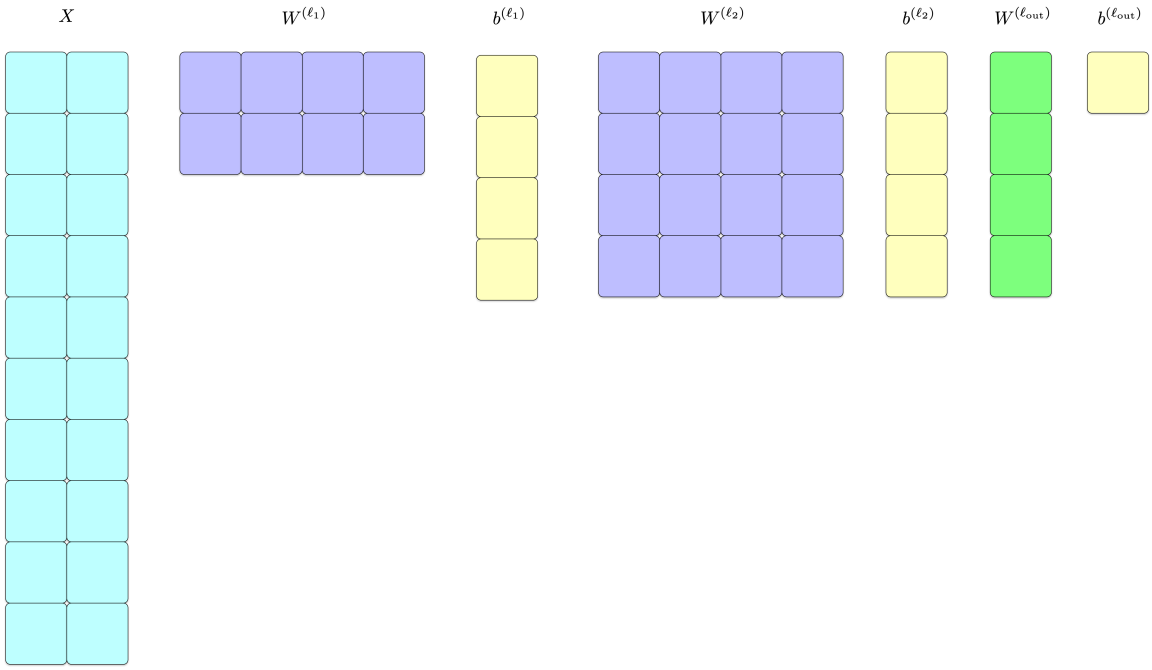
La figure illustre comment la diffusion (*broadcasting*) de NumPy ajoute le vecteur de biais $b^{(\ell_1)}$ à chaque ligne du produit matriciel $XW^{(\ell_1)}$. Les diapositives suivantes présentent les autres calculs intermédiaires. Suivez les dimensions d'entrée et de sortie à chaque étape pour comprendre comment chaque opération transforme les dimensions des matrices.

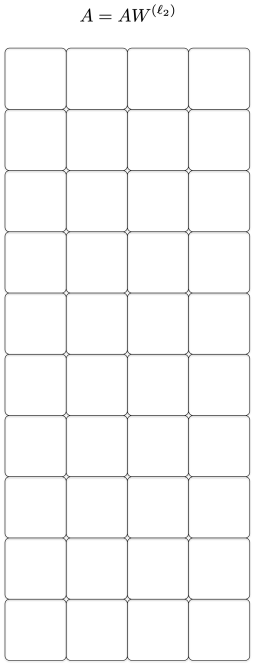
Suivre les dimensions (suite)



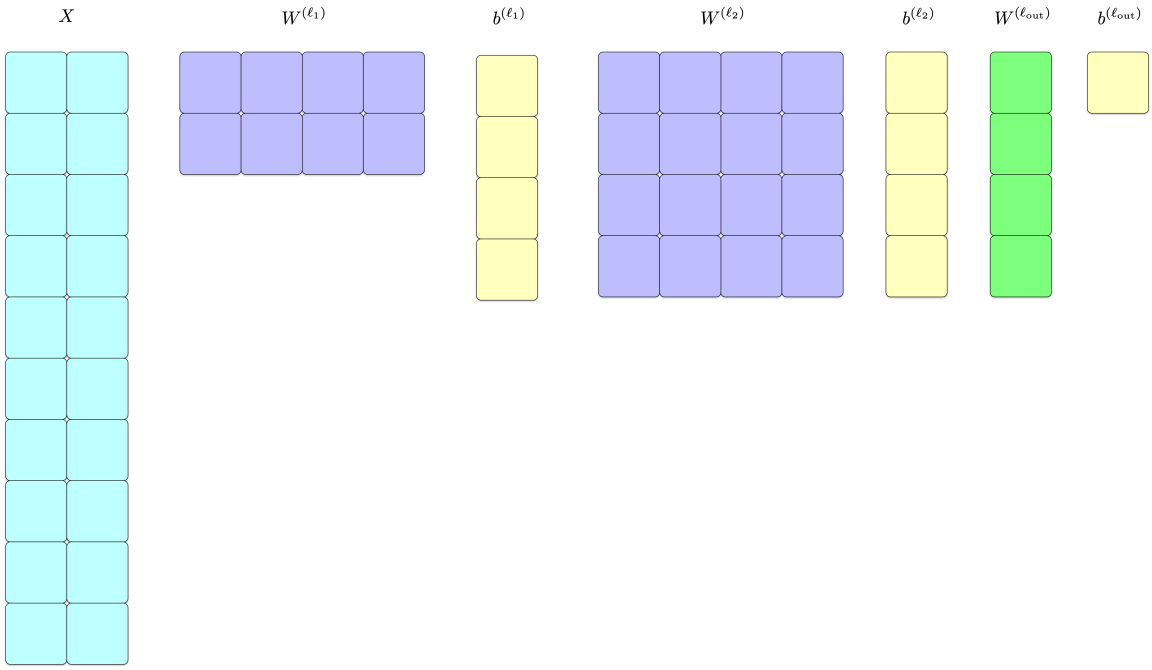


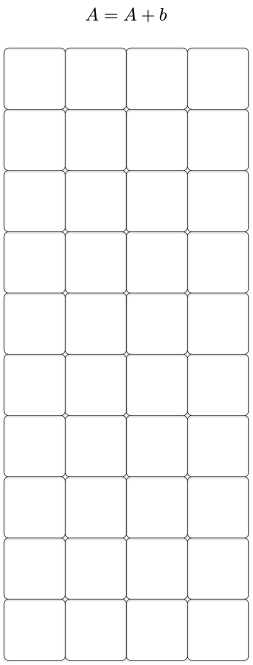
Suivre les dimensions (suite)



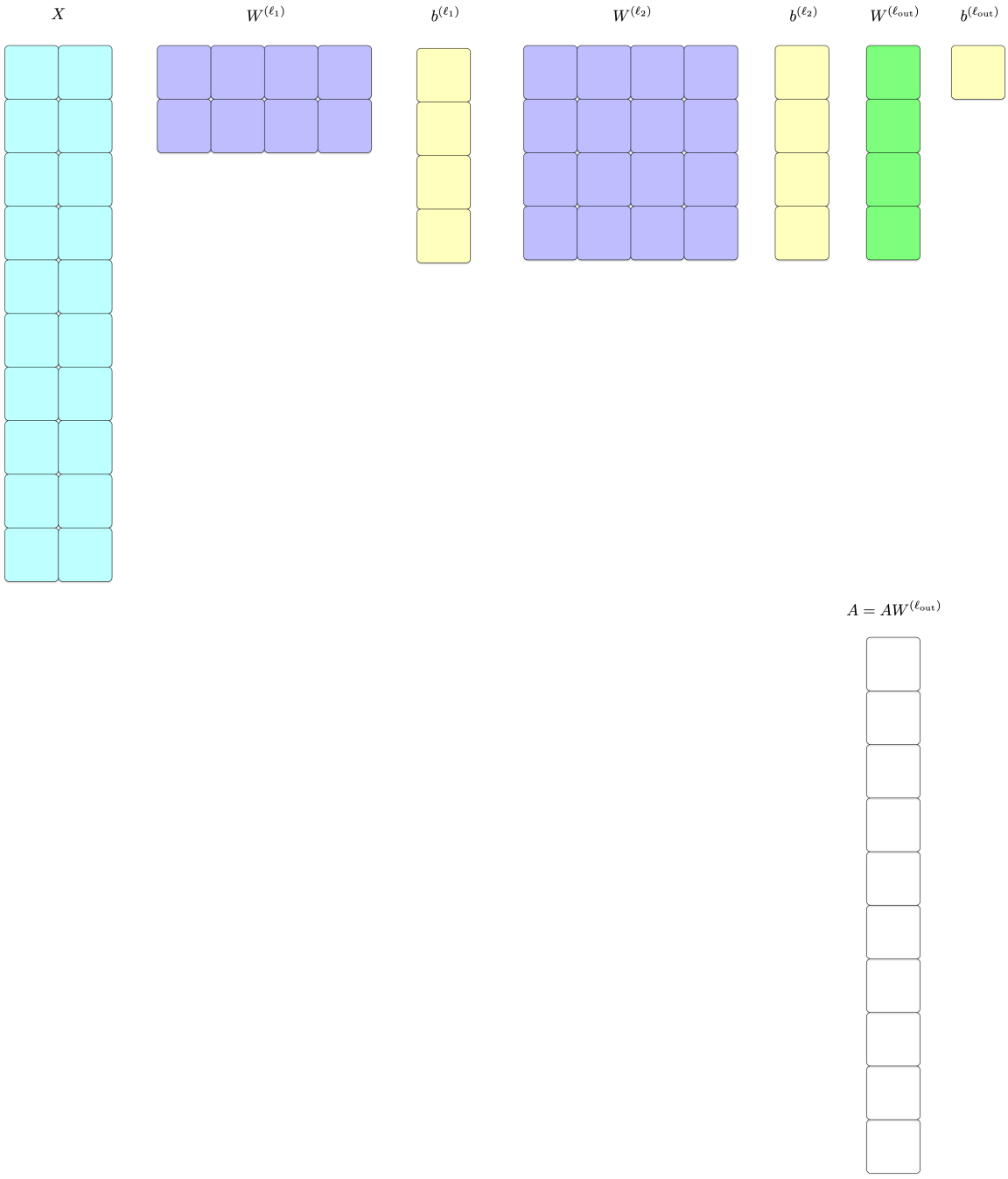


Suivre les dimensions (suite)

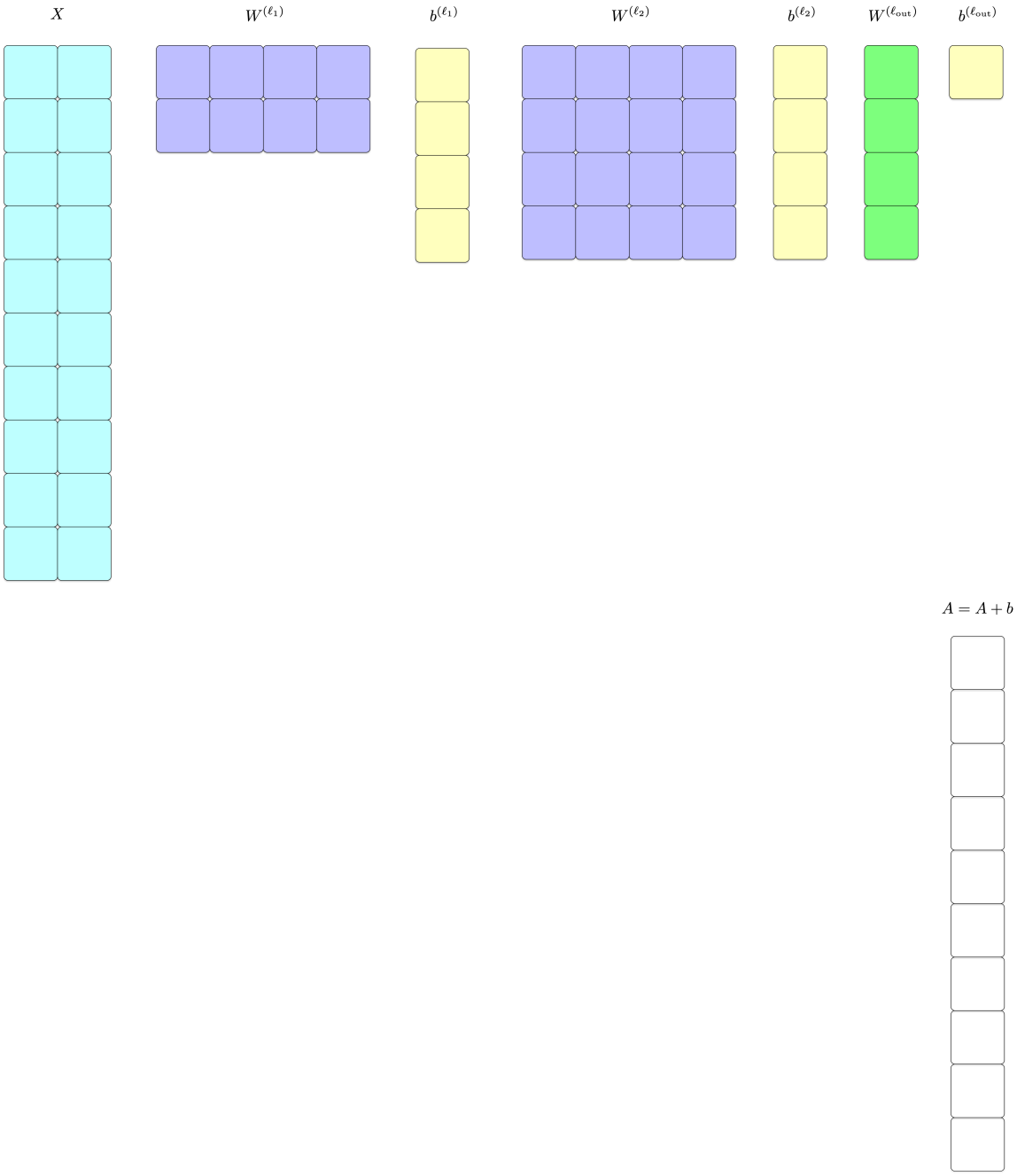




Suivre les dimensions (suite)



Suivre les dimensions (suite)



Faire des prédictions

```
    def predict(self, X, threshold=0.5):

        return (self.forward(X) >= threshold).astype(int)
```

Il s'agit bien de prédictions, au pluriel.

La méthode `forward` produit un tableau de probabilités comprises entre 0 et 1, indiquant pour chaque exemple x_i sa probabilité d'appartenir à la classe 1. L'expression `self.forward(X) >= threshold` produit un tableau de valeurs booléennes : `True` si la probabilité est supérieure ou égale au seuil `threshold`, et `False` sinon. Ces valeurs sont ensuite converties en étiquettes binaires 0 et 1.

Modifier le seuil de décision `threshold` permet d'ajuster le compromis entre précision et rappel.

Probabilités, étiquettes et perte

Pour un exemple dont la véritable étiquette est **1** :

Probabilité prédite	Classe au seuil de 0,5	Entropie croisée binaire
0,51	1	$-\log(0.51) \approx 0.673$
0,99	1	$-\log(0.99) \approx 0.010$

Les deux prédictions sont **correctes**, mais leurs pertes diffèrent.

- `forward(X)` retourne des **probabilités**.
- `predict(X)` retourne des **étiquettes de classe**.
- `loss(X, y)` évalue les **probabilités** par rapport aux cibles.

La perte peut diminuer alors que l'exactitude reste inchangée. Pour un exemple positif, faire passer la probabilité prédite de 0,51 à 0,99 réduit l'entropie croisée binaire, même si les deux valeurs prédisent la classe 1. À l'inverse, une prédiction incorrecte faite avec une grande confiance entraîne une perte plus élevée.

Il s'agit de la même entropie croisée binaire, ou moyenne de la log-vraisemblance négative, utilisée pour la régression logistique dans la présentation 6. La sortie du réseau de neurones remplace la probabilité calculée par le modèle de régression logistique; la définition de la perte reste inchangée.

Calculer la perte

```
    def loss(self, X, y):

        return bce_loss(y, self.forward(X))
```

Question de révision : Pourquoi utilisons-nous `bce_loss(y, self.forward(X))` plutôt que `bce_loss(y, self.predict(X))` ?

La méthode calcule la moyenne de l'entropie croisée binaire sur le jeu de données.

Réponse : `forward(X)` retourne des probabilités, tandis que `predict(X)` les convertit en étiquettes de classe. Les probabilités permettent à la perte de reconnaître les changements de confiance, même si la classe prédite reste la même. Par exemple, pour un exemple positif, 0,51 et 0,99 prédisent tous deux la classe 1, mais 0,99 donne une perte plus faible. Le bornage empêche `log(0)` ; il ne peut pas rétablir l'information de confiance éliminée par l'application du seuil.

Discussion

À l'exception de l'algorithme d'entraînement, l'implémentation de notre réseau de neurones est maintenant complète.

Sans faire appel à la rétropropagation, comment proposeriez-vous d'apprendre les paramètres du modèle?

. . .

Essayer des modifications de paramètres → retenir les meilleures modifications individuelles → les appliquer ensemble → répéter.

Pseudocode

```
pseudocode
pour chaque itération :
    current_loss = compute_loss(network, training_data)
    deltas = {}
    pour chaque paramètre p du réseau :  # poids ET biais
        original_value = value(p)
        best_delta = 0
        best_loss = current_loss
        pour delta dans [-step, 0, +step] :
            loss_temp = compute_loss_with_replacement(
                p, original_value + delta, training_data)
            si loss_temp < best_loss :
                best_loss = loss_temp
                best_delta = delta
        deltas[p] = best_delta

    # Appliquer toutes les modifications retenues
simultanément
    pour chaque paramètre p du réseau :
        value(p) += deltas[p]
```

Cet algorithme choisit les mises à jour en **essayant des modifications de paramètres**. La rétropropagation calculera des **gradients**, qu'un optimiseur utilisera pour choisir les mises à jour. Les deux s'intègrent dans une boucle d'entraînement qui évalue le réseau et met ses paramètres à jour.

Chaque modification candidate est évaluée en gardant tous les autres paramètres à leur valeur du début de l'itération. Le remplacement temporaire est annulé après l'essai. Les modifications retenues sont appliquées ensemble seulement après l'examen de tous les paramètres. Dans le code, `epochs` compte ces parcours complets des paramètres; chaque parcours évalue l'ensemble d'entraînement à de nombreuses reprises.

Une modification qui réduit la perte lorsqu'elle est appliquée seule peut interagir défavorablement avec les modifications d'autres paramètres. L'application simultanée de toutes les modifications retenues peut donc annuler de bons choix individuels et augmenter la perte totale. Ni une diminution de la perte ni une augmentation de l'exactitude ne sont garanties à chaque itération.

La structure simultanée relie cet exemple à l'entraînement fondé sur les gradients : l'information nécessaire à toutes les mises à jour est recueillie avec les paramètres courants, puis les mises à jour sont appliquées. La prochaine présentation remplace les essais individuels par les gradients calculés au moyen de la rétropropagation.

Détour Python : parcourir les paramètres

Un court détour Python : les **générateurs**, les **repères** (*tags*) des paramètres et la lecture ou la modification d'éléments de tableaux.

```
    def _all_param_tags(self):

        """
        Produit des repères pour chaque paramètre scalaire :
        ('W', layer_idx, i, j) ou ('b', layer_idx, j)
        """

        for l, W in enumerate(self.W):
            for i in range(W.shape[0]):
                for j in range(W.shape[1]):
                    yield ('W', l, i, j)
            for j in range(self.b[l].shape[0]):
                yield ('b', l, j)
```

`_all_param_tags` est un générateur : il fournit l'emplacement de chaque poids et de chaque biais scalaire sans construire d'abord une liste distincte. Les exemples suivants expliquent le mécanisme Python utilisé dans la boucle d'entraînement.

Détour Python : générateurs

Un **générateur** produit des valeurs une à la fois, s'arrête à `yield` et reprend lorsqu'on demande la valeur suivante.

In [9]:
```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

In [10]:
```
c = countdown(3)
print(next(c))  # 3
print(next(c))  # 2
print(next(c))  # 1
```

```
3
2
1
```

L'appel `countdown(3)` crée un générateur; il n'exécute pas immédiatement le corps de la fonction. Chaque appel à `next(c)` reprend l'exécution jusqu'au prochain `yield`. L'état local, y compris `n`, est conservé entre les appels.

Détour Python : utiliser un itérateur

Après sa dernière valeur, le générateur lève l'exception `StopIteration` :

In [11]:
```
try:
    print(next(c))
except StopIteration:
    print("Exception StopIteration interceptée.")
```

```
Exception StopIteration interceptée.
```

Une boucle `for` gère automatiquement cette fin d'itération :

In [12]:
```
for value in countdown(3):
    print(value)
```

```
3
2
1
```

La boucle `for` demande des valeurs successives jusqu'à l'épuisement de l'itérateur. Il n'est pas nécessaire d'intercepter explicitement `StopIteration` dans cette forme.

Les fonctions intégrées `enumerate` et `zip` retournent aussi des itérateurs. `enumerate(self.W)` fournit l'indice d'une couche et sa matrice de poids, tandis que `zip(self.W, self.b)` associe chaque matrice à son vecteur de biais. Le générateur de paramètres utilise des boucles imbriquées pour produire un repère pour chaque poids et chaque biais.

Détour Python : repères des paramètres

In [13]:
```
class Demo:

    def __init__(self, layer_sizes):
        self.sizes = list(layer_sizes)
        rng = np.random.default_rng(0)
        self.W = [rng.standard_normal(size=(in_d, out_d)) * 0.5
                  for in_d, out_d in zip(layer_sizes[:-1], layer_sizes[1:])]
        self.b = [np.zeros(out_d) for out_d in layer_sizes[1:]]

    def _all_param_tags(self):

        """
```

```
        Produit des repères pour chaque paramètre scalaire :
        ('W', layer_idx, i, j) ou ('b', layer_idx, j)
        """

        for l, W in enumerate(self.W):
            for i in range(W.shape[0]):
                for j in range(W.shape[1]):
                    yield ('W', l, i, j)
            for j in range(self.b[l].shape[0]):
                yield ('b', l, j)

    def show(self):

      for tag in self._all_param_tags():
        print(tag)
```

In [14]:
```
d = Demo([2,4,4,1])
d.show()
```

```
('W', 0, 0, 0)
('W', 0, 0, 1)
('W', 0, 0, 2)
('W', 0, 0, 3)
('W', 0, 1, 0)
('W', 0, 1, 1)
('W', 0, 1, 2)
('W', 0, 1, 3)
('b', 0, 0)
('b', 0, 1)
('b', 0, 2)
('b', 0, 3)
('W', 1, 0, 0)
('W', 1, 0, 1)
('W', 1, 0, 2)
('W', 1, 0, 3)
('W', 1, 1, 0)
('W', 1, 1, 1)
('W', 1, 1, 2)
('W', 1, 1, 3)
('W', 1, 2, 0)
('W', 1, 2, 1)
('W', 1, 2, 2)
('W', 1, 2, 3)
('W', 1, 3, 0)
('W', 1, 3, 1)
('W', 1, 3, 2)
('W', 1, 3, 3)
('b', 1, 0)
('b', 1, 1)
('b', 1, 2)
('b', 1, 3)
('W', 2, 0, 0)
('W', 2, 1, 0)
('W', 2, 2, 0)
('W', 2, 3, 0)
('b', 2, 0)
```

Détour Python : lire un paramètre

```
    def _get_param(self, tag):
        kind = tag[0]
        if kind == 'W':
            _, l, i, j = tag
            return self.W[l][i, j]
        else:
            _, l, j = tag
            return self.b[l][j]
```

À partir d'un repère `tag`, soit `('W', l, i, j)`, soit `('b', l, j)`, la méthode récupère le poids ou le terme de biais correspondant aux indices fournis. `l` désigne la **couche**; `i` et `j` sont les indices de ligne et de colonne de `W[l]`, ou `j` est un indice de `b[l]`.

Détour Python : modifier un paramètre

```
    def _set_param(self, tag, val):
        kind = tag[0]
        if kind == 'W':
            _, l, i, j = tag
            self.W[l][i, j] = val
        else:
            _, l, j = tag
            self.b[l][j] = val
```

La logique est semblable, mais la méthode modifie `self.W[l][i, j]` ou `self.b[l][j] = val`, selon le type de repère.

Entraînement (apprentissage)

Le détour Python est terminé. Ces fonctions auxiliaires permettent à la boucle d'entraînement d'essayer des modifications et de mettre à jour **chaque poids et chaque biais**.

```
    def train(self, X, y, epochs=10, verbose=True):

        for ep in range(1, epochs + 1):

            base_loss = self.loss(X, y)
            updates = {}

            # Essayer des modifications de tous les paramètres
            for tag in self._all_param_tags():

                theta = self._get_param(tag)
                best_delta = 0.0
                best_loss = base_loss
```

```
                for delta in (-self.step, 0.0, +self.step):
                    self._set_param(tag, theta + delta)
                    trial_loss = self.loss(X, y)
                    if trial_loss < best_loss:
                        best_loss = trial_loss
                        best_delta = delta

                # rétablir la valeur d'origine
                self._set_param(tag, theta)
                updates[tag] = best_delta

            # Appliquer toutes les modifications ensemble
            for tag, d in updates.items():
                if d != 0.0:
                    self._set_param(tag, self._get_param(tag) + d)

            new_loss = self.loss(X, y)

            if verbose:
                print(f"Époque {ep:3d} : perte {base_loss:.5f} → {new_loss:.5f}")

            # arrêt anticipé facultatif
            if abs(new_loss - base_loss) < 1e-12:
                break
```

Les essais utilisent les mêmes paramètres de référence, et chaque paramètre est rétabli avant l'examen du suivant. Le dictionnaire conserve les modifications retenues; la seconde boucle les applique simultanément. Comme les interactions sont ignorées pendant la sélection, la mise à jour combinée peut augmenter la perte, même si chaque modification non nulle était avantageuse lorsqu'elle était appliquée seule.

La condition d'arrêt vérifie si la variation de la perte d'entraînement est très petite. Il s'agit d'une règle d'arrêt simple pour la démonstration, et non d'une garantie de solution optimale ou de bonne généralisation.

Entraînement sur les cercles

Est-ce que cela fonctionne?

In [15]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = NaiveMLP([2, 4, 4, 1], step=0.06, seed=0)

print("Perte initiale :", model.loss(X_train, y_train))

model.train(X_train, y_train, epochs=100)
```

```
print("Exactitude (entraînement) :", accuracy_score(y_train, model.predict(X
print("Exactitude (test) : ", accuracy_score(y_test, model.predict(X_test)))
```

```
Perte initiale : 0.7001969055705487
Époque   1 : perte 0.70020 → 0.69315
Époque   2 : perte 0.69315 → 0.69547
Époque   3 : perte 0.69547 → 0.69387
Époque   4 : perte 0.69387 → 0.69542
Époque   5 : perte 0.69542 → 0.69384
Époque   6 : perte 0.69384 → 0.69537
Époque   7 : perte 0.69537 → 0.69382
Époque   8 : perte 0.69382 → 0.69531
Époque   9 : perte 0.69531 → 0.69379
Époque  10 : perte 0.69379 → 0.69525
Époque  11 : perte 0.69525 → 0.69361
Époque  12 : perte 0.69361 → 0.69517
Époque  13 : perte 0.69517 → 0.69331
Époque  14 : perte 0.69331 → 0.69503
Époque  15 : perte 0.69503 → 0.69298
Époque  16 : perte 0.69298 → 0.69468
Époque  17 : perte 0.69468 → 0.69257
Époque  18 : perte 0.69257 → 0.69364
Époque  19 : perte 0.69364 → 0.69200
Époque  20 : perte 0.69200 → 0.69173
Époque  21 : perte 0.69173 → 0.69118
Époque  22 : perte 0.69118 → 0.68965
Époque  23 : perte 0.68965 → 0.68970
Époque  24 : perte 0.68970 → 0.68716
Époque  25 : perte 0.68716 → 0.68672
Époque  26 : perte 0.68672 → 0.68417
Époque  27 : perte 0.68417 → 0.68170
Époque  28 : perte 0.68170 → 0.67855
Époque  29 : perte 0.67855 → 0.67406
Époque  30 : perte 0.67406 → 0.66867
Époque  31 : perte 0.66867 → 0.66203
Époque  32 : perte 0.66203 → 0.65492
Époque  33 : perte 0.65492 → 0.64672
Époque  34 : perte 0.64672 → 0.63852
Époque  35 : perte 0.63852 → 0.62948
Époque  36 : perte 0.62948 → 0.61977
Époque  37 : perte 0.61977 → 0.60918
Époque  38 : perte 0.60918 → 0.59844
Époque  39 : perte 0.59844 → 0.58893
Époque  40 : perte 0.58893 → 0.57598
Époque  41 : perte 0.57598 → 0.56310
Époque  42 : perte 0.56310 → 0.55035
Époque  43 : perte 0.55035 → 0.53809
Époque  44 : perte 0.53809 → 0.52214
Époque  45 : perte 0.52214 → 0.50660
Époque  46 : perte 0.50660 → 0.49073
Époque  47 : perte 0.49073 → 0.47591
Époque  48 : perte 0.47591 → 0.45758
Époque  49 : perte 0.45758 → 0.44074
Époque  50 : perte 0.44074 → 0.42251
Époque  51 : perte 0.42251 → 0.41069
Époque  52 : perte 0.41069 → 0.38858
Époque  53 : perte 0.38858 → 0.36998
Époque  54 : perte 0.36998 → 0.35227
Époque  55 : perte 0.35227 → 0.34356
```

```
Époque  56 : perte 0.34356 → 0.32577
Époque  57 : perte 0.32577 → 0.31462
Époque  58 : perte 0.31462 → 0.29240
Époque  59 : perte 0.29240 → 0.27704
Époque  60 : perte 0.27704 → 0.25851
Époque  61 : perte 0.25851 → 0.25409
Époque  62 : perte 0.25409 → 0.23884
Époque  63 : perte 0.23884 → 0.23260
Époque  64 : perte 0.23260 → 0.21815
Époque  65 : perte 0.21815 → 0.21238
Époque  66 : perte 0.21238 → 0.19964
Époque  67 : perte 0.19964 → 0.19350
Époque  68 : perte 0.19350 → 0.17787
Époque  69 : perte 0.17787 → 0.16963
Époque  70 : perte 0.16963 → 0.15270
Époque  71 : perte 0.15270 → 0.14804
Époque  72 : perte 0.14804 → 0.13478
Époque  73 : perte 0.13478 → 0.13357
Époque  74 : perte 0.13357 → 0.12381
Époque  75 : perte 0.12381 → 0.12041
Époque  76 : perte 0.12041 → 0.10789
Époque  77 : perte 0.10789 → 0.10512
Époque  78 : perte 0.10512 → 0.09204
Époque  79 : perte 0.09204 → 0.08493
Époque  80 : perte 0.08493 → 0.07447
Époque  81 : perte 0.07447 → 0.07243
Époque  82 : perte 0.07243 → 0.06423
Époque  83 : perte 0.06423 → 0.06479
Époque  84 : perte 0.06479 → 0.06053
Époque  85 : perte 0.06053 → 0.05940
Époque  86 : perte 0.05940 → 0.05501
Époque  87 : perte 0.05501 → 0.05465
Époque  88 : perte 0.05465 → 0.05370
Époque  89 : perte 0.05370 → 0.05036
Époque  90 : perte 0.05036 → 0.04376
Époque  91 : perte 0.04376 → 0.04596
Époque  92 : perte 0.04596 → 0.04283
Époque  93 : perte 0.04283 → 0.04213
Époque  94 : perte 0.04213 → 0.04031
Époque  95 : perte 0.04031 → 0.03879
Époque  96 : perte 0.03879 → 0.03629
Époque  97 : perte 0.03629 → 0.03574
Époque  98 : perte 0.03574 → 0.03499
Époque  99 : perte 0.03499 → 0.03293
Époque 100 : perte 0.03293 → 0.03075
Exactitude (entraînement) : 1.0
Exactitude (test) :  1.0
```

Ces résultats illustrent que l'algorithme naïf peut apprendre une frontière de décision utile. Ils ne garantissent pas le succès avec d'autres initialisations, tailles de pas ou jeux de données. Gardez la partition de test à l'écart de l'entraînement et du choix des réglages; les données de validation conviennent à la sélection de ces réglages. Les valeurs fixes de `step` et de `seed` rendent cette démonstration reproductible.

Visualisation

In [16]:

```
# Tracer la frontière de décision dans le plan (x1, x2) d'origine

def plot_decision_boundary(model, X, y, title="Frontière de décision du MLP

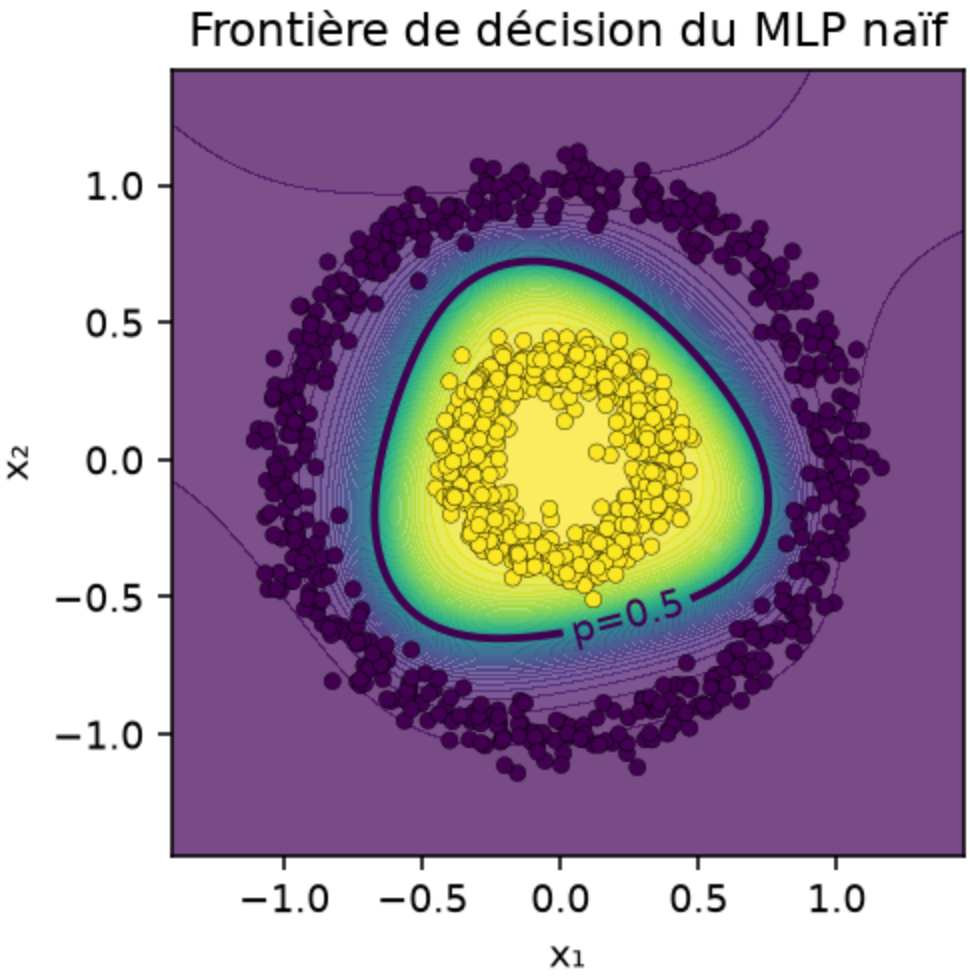
    # grille dans le plan des entrées
    pad = 0.3
    x1_min, x1_max = X[:,0].min()-pad, X[:,0].max()+pad
    x2_min, x2_max = X[:,1].min()-pad, X[:,1].max()+pad

    xx, yy = np.meshgrid(
        np.linspace(x1_min, x1_max, 400),
        np.linspace(x2_min, x2_max, 400)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]

    # prédire les probabilités sur la grille
    p = model.forward(grid).reshape(xx.shape)

    # probabilités, contour p=0,5 et exemples
    plt.figure(figsize=(3.75, 3.75), dpi=140)
    plt.contourf(xx, yy, p, levels=50, alpha=0.7)
    cs = plt.contour(xx, yy, p, levels=[0.5], linewidths=2)
    plt.scatter(X[:,0], X[:,1], c=y, s=18, edgecolor="k", linewidth=0.2)
    plt.clabel(cs, fmt={0.5: "p=0.5"})
    plt.title(title)
    plt.xlabel("x₁")
    plt.ylabel("x₂")
    plt.tight_layout()
    plt.show()

plot_decision_boundary(model, X, y)
```



Données de type XNOR

In [17]:

```
n_samples = 800
rng = np.random.default_rng(42)

X = rng.uniform(-6, 6, size=(n_samples, 2))
x1, x2 = X[:, 0], X[:, 1]

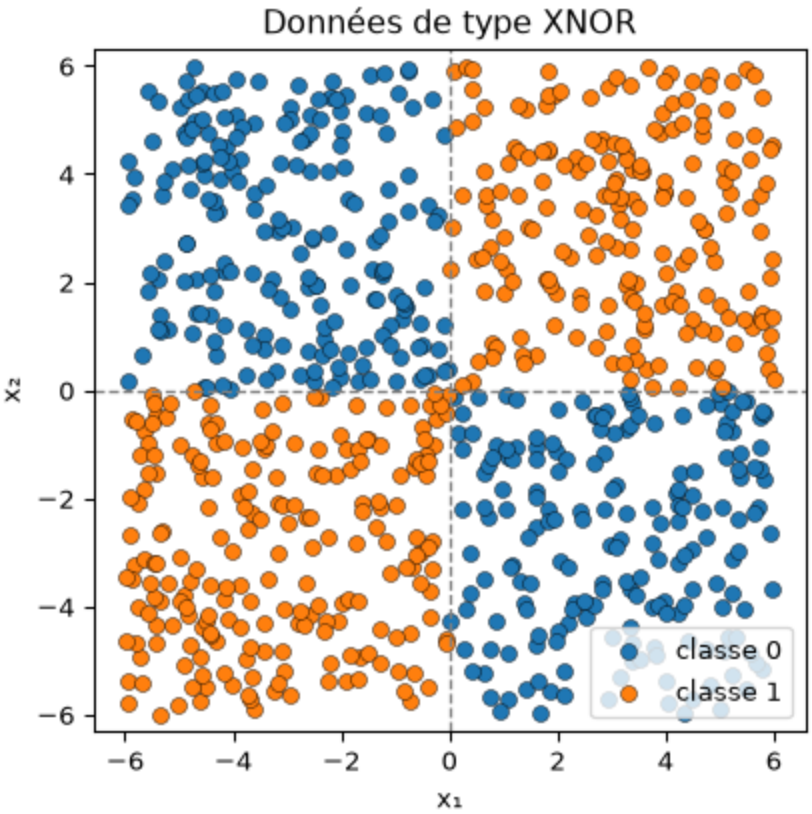
y = ((x1 * x2) > 0).astype(int)

plt.figure(figsize=(4.5, 4.5))
plt.scatter(X[y == 0, 0], X[y == 0, 1],
            color="C0", label="classe 0", edgecolor="k", linewidth=0.3)
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            color="C1", label="classe 1", edgecolor="k", linewidth=0.3)

plt.axhline(0, color="gray", linestyle="--", linewidth=1)
plt.axvline(0, color="gray", linestyle="--", linewidth=1)

plt.xlabel("x₁")
plt.ylabel("x₂")
plt.title("Données de type XNOR")
plt.xlim(-6, 6)
```

```
plt.ylim(-6, 6)
plt.axis("equal")
plt.legend()
plt.tight_layout()
plt.show()
```



XNOR est le complément de XOR : la classe 1 correspond aux entrées échantillonnées dont les signes concordent. Les deux problèmes exigent une frontière de décision non linéaire. La graine aléatoire et la taille du pas ont été choisies pour fournir une démonstration claire et reproductible; l'algorithme naïf reste sensible à ces choix.

L'architecture reste `[2, 4, 4, 1]`. L'entraînement détermine des valeurs de paramètres différentes pour ce nouveau jeu de données.

Données de type XNOR (suite)

In [18]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = NaiveMLP([2, 4, 4, 1], step=0.06, seed=0)

print("Perte initiale :", model.loss(X_train, y_train))

model.train(X_train, y_train, epochs=100, verbose=False)

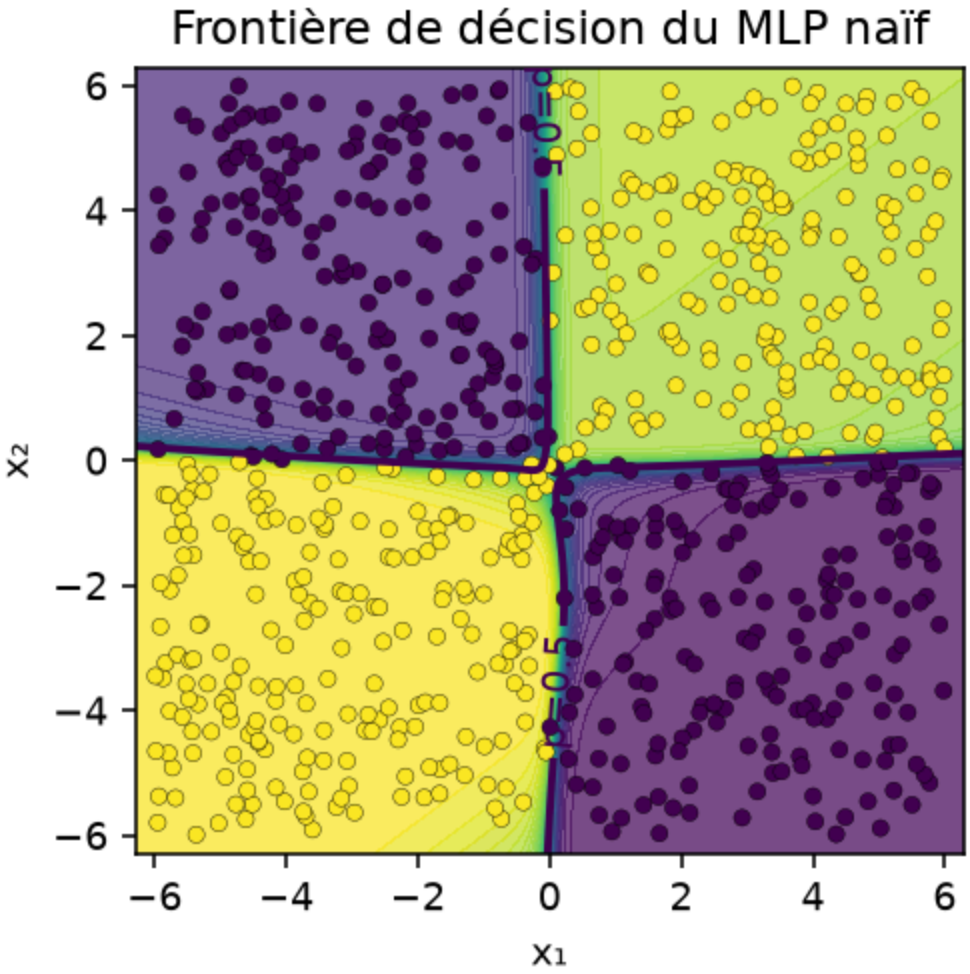
print("Perte finale :", model.loss(X_train, y_train))
```

```
print("Exactitude (entraînement) :", accuracy_score(y_train, model.predict(X
print("Exactitude (test) : ", accuracy_score(y_test, model.predict(X_test)))
```

```
Perte initiale : 0.7029764953722529
Perte finale : 0.09228049882862682
Exactitude (entraînement) : 0.9866666666666667
Exactitude (test) :  0.98
```

Données de type XNOR (suite)

In [19]:
```
plot_decision_boundary(model, X, y)
```



Notre réseau de neurones et son algorithme d'entraînement naïf ont été évalués sur deux problèmes distincts. Malgré sa simplicité, le modèle a appris des frontières de décision sensiblement différentes sans nécessiter la conception d'attributs supplémentaires.

Pour garder cet exemple simple et clair, nous avons utilisé les données brutes sans mise à l'échelle. En pratique, il est courant de mettre les attributs à l'échelle ou de les normaliser avant d'entraîner un réseau de neurones ou un modèle optimisé par une méthode fondée sur les gradients. La mise à l'échelle favorise une convergence plus

rapide et plus stable, mais ajoute des étapes de prétraitement et de transformation. L'objectif étant ici de comprendre le mécanisme d'entraînement, nous l'omettons dans cet exemple.

Avant l'entraînement :

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Pour tracer la frontière dans l'espace d'attributs d'origine, construisez la grille dans les coordonnées d'origine et passez `scaler.transform(grid)` à `model.forward`. Les prédictions peuvent ensuite être tracées sur la grille d'origine inchangée. Le modèle doit recevoir des entrées transformées de la même manière que ses données d'entraînement.

Limites

- **Évaluations répétées :** chaque essai de modification recalcule les prédictions et la perte sur l'ensemble d'entraînement.
- **Taille de pas fixe :** les seules modifications possibles sont `-step`, `0` et `+step`.
- **Interactions entre paramètres :** des modifications utiles individuellement peuvent entrer en conflit lorsqu'elles sont appliquées ensemble.
- **Sensibilité :** les résultats dépendent de l'initialisation, de la taille du pas et du jeu de données.

La méthode apprend en comparant des pertes. Ces comparaisons fournissent une certaine information sur la direction des modifications essayées, mais elles ne calculent pas de gradient et n'utilisent pas son amplitude pour choisir une mise à jour.

Un petit pas fixe peut ralentir les progrès, tandis qu'un grand pas peut sauter des valeurs utiles des paramètres ou provoquer des oscillations. Comme toutes les modifications retenues sont appliquées ensemble, la perte peut augmenter même si les essais individuels suggéraient des améliorations.

La généralisation exige toujours une évaluation distincte, comme pour l'entraînement fondé sur les gradients. Le surapprentissage et le besoin de régularisation ne sont pas des limites propres à cette méthode naïve.

Pourquoi une méthode plus rapide est nécessaire

Notre petit réseau possède **37 paramètres**.

- Trois essais par paramètre : $3 \times 37 = 111$ évaluations de la perte.
- Une évaluation de référence et une évaluation finale : **113 évaluations par parcours**.
- Chaque évaluation applique le réseau à **tout l'ensemble d'entraînement**.

La rétropropagation calcule les gradients de tous les paramètres à l'aide d'une **propagation avant** et d'une **propagation arrière**. Un optimiseur met ensuite les paramètres à jour ensemble.

Ce décompte décrit l'implémentation présentée, y compris les essais sans modification. Il met en évidence le coût des évaluations répétées d'un réseau dont nous comprenons déjà le calcul. Pour un réseau comportant beaucoup de paramètres, répéter des évaluations complètes pour chaque paramètre devient coûteux.

La rétropropagation réutilise des résultats intermédiaires pour calculer les gradients efficacement. Son coût augmente toujours avec la taille du réseau; elle évite une évaluation complète distincte pour chaque paramètre scalaire. Une mise à jour fondée sur les gradients exige aussi un taux d'apprentissage approprié et ne garantit pas que chaque mise à jour réduise la perte.

Conclusion

Résumé

- Implémenté un **réseau à propagation avant complet** avec des tableaux NumPy.
- Suivi les **dimensions** des **entrées**, des **poids**, des **biais** et des **sorties des couches**.
- Distingué les **probabilités prédites**, les **étiquettes de classe** et l'**entropie croisée binaire**.
- Utilisé une **méthode naïve sans dérivées** pour choisir des modifications de paramètres et les appliquer simultanément.
- Relié les **évaluations répétées de la perte** et les **interactions entre paramètres** au besoin d'un calcul efficace des gradients.

Prochaine présentation

- Notation des couches, règle de dérivation en chaîne et rétropropagation.
- Entraînement fondé sur les gradients et comportement des dérivées des fonctions d'activation.

Références

Russell, Stuart, et Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4e éd. Pearson. http://aima.cs.berkeley.edu/.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

École de génie électrique et d'**informatique**

Université d'Ottawa